

Cookie / Session / Token 인증 방식 종류

보통 서버가 클라이언트 인증을 확인하는 방식은 대표적으로 쿠키, 세션, 토큰 3가지 방식이 있다.

JWT를 배우기 앞서 우선 쿠키와 세션의 통신 방식을 복습해보고 이들의 각각 특징과 장단점 그리고 왜 토큰 인증 방식을 사용하느냐에 대해 간단하게 알아본다.

[Cookie 인증]

쿠키는 Key-Value 형식의 문자열 덩어리이다.

클라이언트가 어떠한 웹사이트를 방문할 경우, 그 사이트가 사용하고 있는 서버를 통해 클라이언트의 브라우저에 설치되는 작은 기록 정보 파일이다. 각 사용자마다의 브라우저에 정보를 저장하니 고유 정보 식별이 가능한 것이다.

▼ Request Headers

```
:authority: www.geeksforgeeks.org
:method: GET
:path: /http-headers-cookie/
:scheme: https
accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3
accept-encoding: gzip, deflate, br
accept-language: en-US,en;q=0.9
cache-control: max-age=0
cookie: 6_ENABLED_IDPS=google; _ga=GA1.2.236891924.1569526010; __gads=ID=f8deb276b85d6f74:T=1569559579:S=ALNI_Ma9kGxkZV0d23UT2B6UIi0-iHksPw; __utms=245605906.1569597787.4.4.utmcsr=google|utmccn=(organic)|utmcmd=organic|utmctr=(not%20provided); __utma=245605906.236891924.1569526010.1569592113.1569597786.4; geeksforgeeks_consent_status=dismiss; gfguserName=Sabya_Samadder%2FeyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJpc3MiOiJodHRwczpcL1wvd3d3Lmd1Zm9yZ2V1a3Mub3JnXC8iLCJpYXQiOiE1NzIzNDM3NjAsImV4cCI6MTU3NDkzNTc2MCwiaGFuZGx1Ijo1U2FieWFFuZ2FyYWRkZXIiLCJ1dWlkIjo1YjA2NzA0Njc3MDMzMmY4Y2EyMDcxMDM4OWJjMmVhbnNjEiFQ.f5Bky9sh46uX53XGJupTHQPhSvjgr9_MCV5whZFzkBSCEZR_n4w-5Imj5eS a7TXuFS1r6NI1_VRnuz7Au0P_H-u6SEATsOVskhsEMx0L6oj5NDQw1jk3ZAreK7dk_xyRLgnHsTJws40GbLi9__Yirrp9q2BNGztaMTtXsqri9knMasOVKNmhEGM7bh9fps EYh3PqRRag4WIdGRaZ26Y-orBA91Srj9oyzqVOOFK3zmXd9pHKW7b_ffH5sheGW2EM7uwtjoMiGA7oc6RuG0G8sdpPPYL6Ktfkai2g_oHPRahorsZ_UUQT3jNY91bHt75gx i2PMhw6KOUSNcuJA; wordpress_test_cookie=WP+Cookie+check; AKA_A2=A
referer: https://www.google.com/
sec-fetch-mode: navigate
sec-fetch-site: cross-site
sec-fetch-user: ?1
upgrade-insecure-requests: 1
user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/77.0.3865.120 Safari/537.36
```



브라우저(클라이언트)가 서버에 요청(접속)을 보낸다.

서버는 클라이언트의 요청에 대한 응답을 작성할 때, 클라이언트 측에 저장하고 싶은 정보를 응답 헤더의 Set-Cookie에 담는다. 이후 해당 클라이언트는 요청을 보낼 때마다, 매번 저장된 쿠키를 요청 헤더의 Cookie에 담아 보낸다.

서버는 쿠키에 담긴 정보를 바탕으로 해당 요청의 클라이언트가 누군지 식별하거나 정보를 바탕으로 추천 광고를 띄우거나 한다.

- Cookie 방식의 단점

가장 큰 단점은 **보안에 취약**하다는 점이다.

요청 시 쿠키의 값을 그대로 보내기 때문에 유출 및 조작 당할 위험이 존재한다.

쿠키에는 용량 제한이 있어 많은 정보를 담을 수 없다.

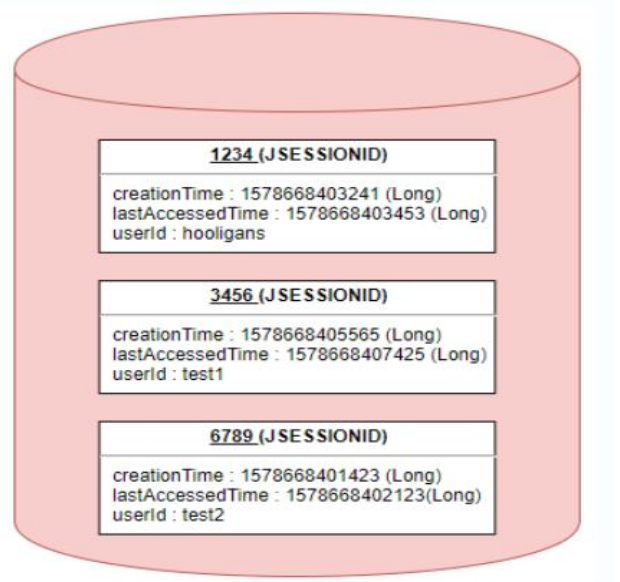
웹 브라우저마다 쿠키에 대한 지원 형태가 다르기 때문에 브라우저간 공유가 불가능하다.

쿠키의 사이즈가 커질수록 네트워크에 부하가 심해진다.

[Session 인증]

쿠키의 보안적인 이슈 때문에, 세션은 비밀번호 등 클라이언트의 민감한 인증 정보를 브라우저가 아닌 서버 측에 저장하고 관리한다. 서버의 메모리에 저장하기도 하고, 서버의 로컬 파일이나 데이터베이스에 저장하기도 한다.

민감한 정보는 클라이언트에 보내지 않고 서버에서 관리한다. 세션 객체는 Key에 해당하는 SESSION ID와 이에 대응하는 Value로 구성되어 있다. Value에는 세션 생성 시간, 마지막 접근 시간 및 User가 저장한 속성 등이 Map 형태로 저장된다.



(1) 유저가 웹사이트에서 로그인하면 세션이 서버 메모리(혹은 데이터베이스) 상에 저장된다.

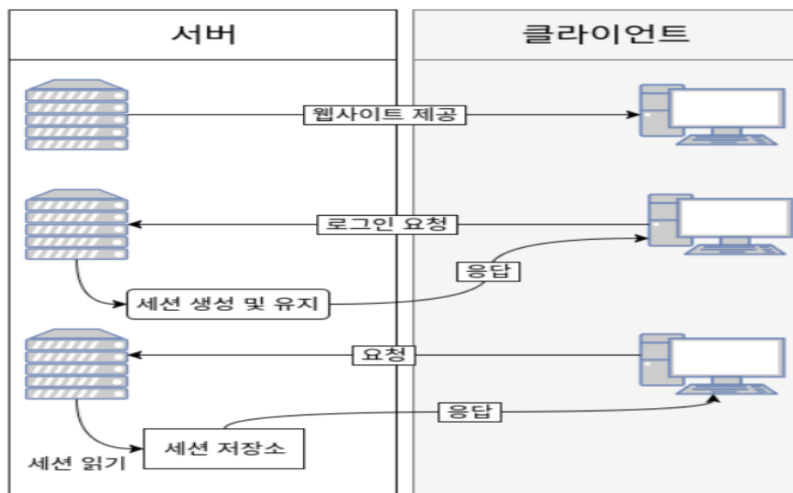
이때, 세션을 식별하기 위한 Session Id를 기준으로 정보를 저장한다.

(2) 서버에서 브라우저에 쿠키에다가 Session ID를 저장한다.

(3) 쿠키에 정보가 담겨있기 때문에 브라우저는 해당 사이트에 대한 모든 Request에 Session Id를

쿠키에 담아 전송한다.

(4) 서버는 클라이언트가 보낸 Session Id 와 서버 메모리로 관리하고 있는 Session ID를 비교하여 인증을 수행한다.



- Session 방식의 단점

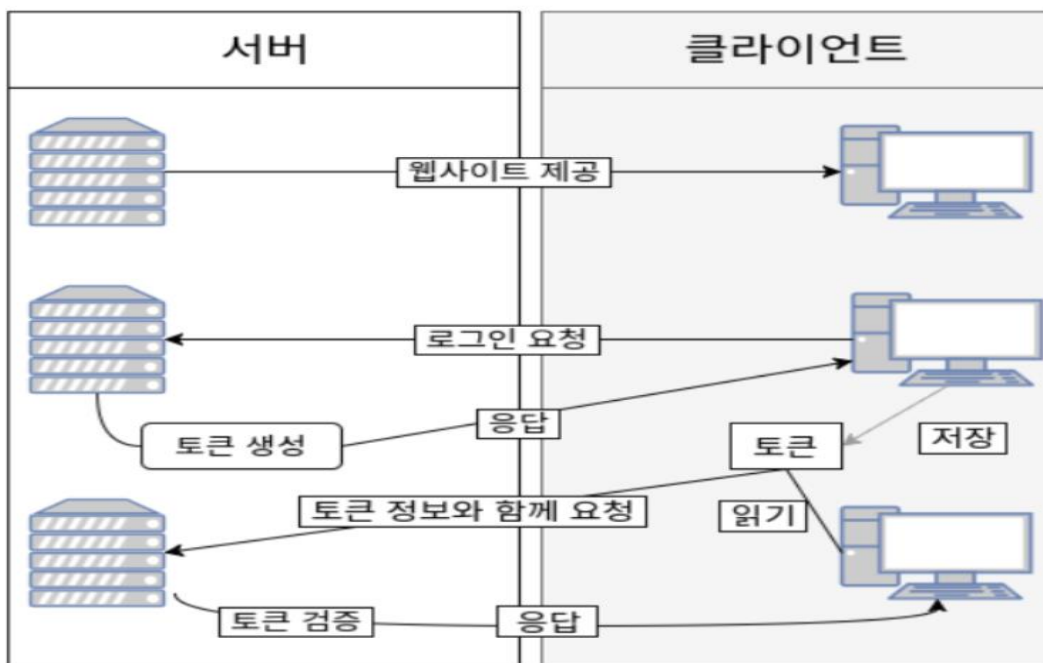
쿠키를 포함한 요청이 외부에 노출되더라도 Session ID 자체는 유의미한 개인정보를 담고 있지 않지만 해커가 Session ID 자체를 탈취하여 클라이언트인척 위장할 수 있다는 한계가 존재한다.

서버에 세션 저장소를 사용하므로 요청이 많아지면 서버에 부하가 심해진다.

[Token 인증]

토큰 기반 인증 시스템은 클라이언트가 서버에 접속을 하면 서버에서 해당 클라이언트에게 인증되었다는 의미로 '토큰'을 부여한다. 이 토큰은 유일하며 토큰을 발급받은 클라이언트는 또 다시 서버에 요청을 보낼 때 **요청 헤더에 토큰을 심어서 보낸다**. 그러면 **서버에서는** 클라이언트로부터 받은 토큰을 **서버에서 제공한 토큰과의 일치 여부를 체크하여 인증 과정을 처리**하게 된다.

기존의 세션기반 인증은 서버가 파일이나 데이터베이스에 세션정보를 가지고 있어야 하고 이를 조회하는 과정이 필요하기 때문에 많은 오버헤드가 발생한다. 하지만 토큰은 세션과는 달리 서버가 아닌 클라이언트에 저장되기 때문에 메모리나 스토리지 등을 통해 세션을 관리했던 서버의 부담을 덜 수 있다. 토큰 자체에 데이터가 들어있기 때문에 클라이언트에서 받아 위조되었는지 판별만 하면 되기 때문이다. **토큰은 앱(app)과 서버가 통신 및 인증할 때 가장 많이 사용된다. 웹에는 쿠키와 세션이 있지만 앱에서는 없기 때문이다.**



- (1) 사용자가 아이디와 비밀번호로 로그인을 한다.
- (2) 서버 측에서 사용자(클라이언트)에게 유일한 **토큰**을 발급한다.
- (3) 클라이언트는 서버 측에서 전달받은 토큰을 **쿠키나 스토리지에 저장**해 두고, 서버에 요청을 할 때마다 해당 토큰을 HTTP 요청 헤더에 포함시켜 전달한다.
- (4) 서버는 전달받은 토큰을 검증하고 요청에 응답한다.
- (5) **토큰에는 요청한 사람의 정보가 담겨**있으므로 서버는 DB를 조회하지 않고 누가 요청하는지 알 수 있다.

- 토큰 기반 인증의 장점

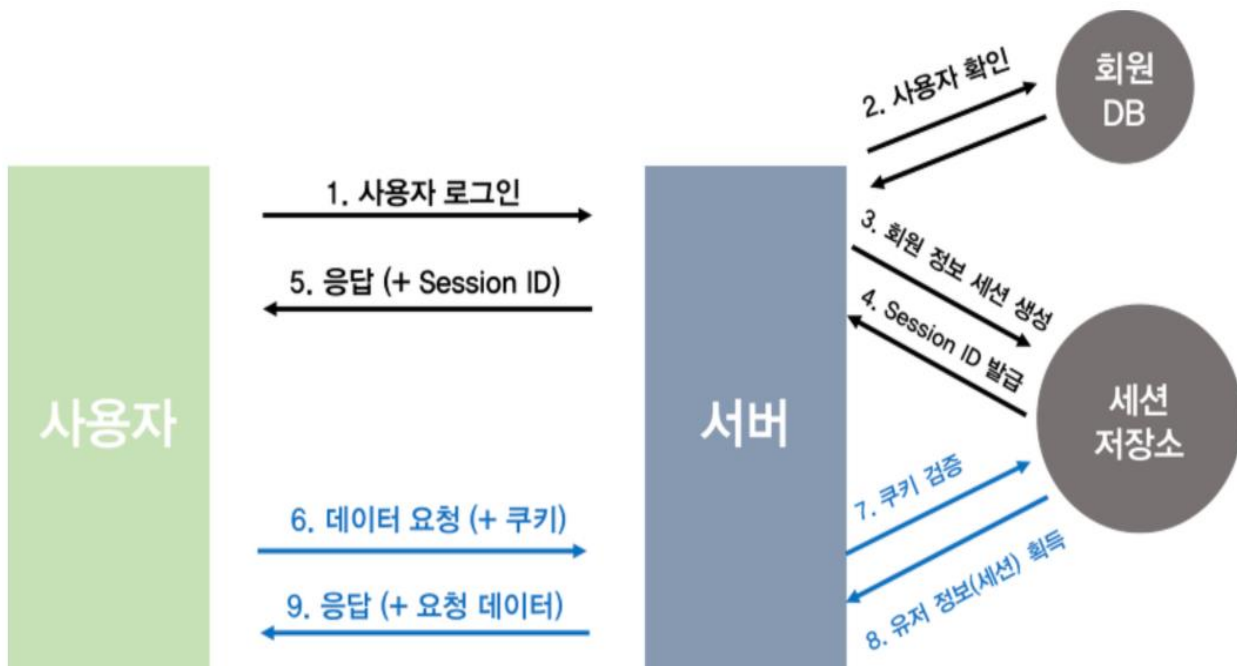
토큰 기반 인증의 장점은 세션 기반 인증의 단점들이 없다는 것이다. 즉, 로그인한 사용자의 상태 정보를 애초에 서버에 저장하지 않으므로 서버 저장소의 오버헤드를 신경 쓸 필요가 없고 복잡

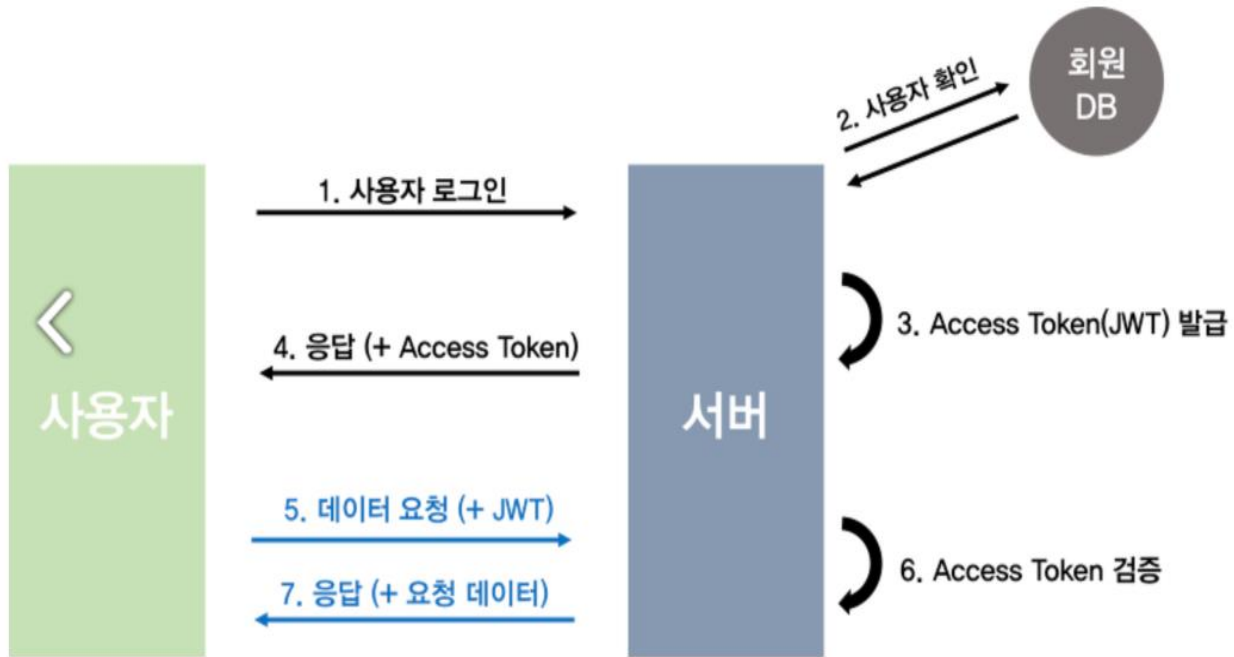
한 구성과 환경에서 로그인한 사용자의 상태 정보를 기억해야 하는 부담도 없으니 서버 확장에 훨씬 유리하다.

- 토큰 기반 인증의 단점

토큰 기반 인증도 단점이 없는 것은 아니다. 서버를 대신해서 클라이언트가 로그인한 사용자의 상태 정보를 기억하는 것이기 때문에, 토큰 자체에 로그인한 사용자의 상태 정보가 담겨 있다. 따라서 상대적으로 서버보다 보안이 튼튼하지 못한 클라이언트에서 토큰이 탈취당하면 몇몇 개인정보까지 탈취될 위험이 있다. 세션 기반 인증과 달리 서버는 이미 한 번 발급한 토큰을 무효화 시킬 방법이 없기 때문에 탈취된 토큰은 그 유효 기한이 지나기 전까지 마음껏 사용할 수 있다. **이러한 단점은 발급된 토큰의 유효시간을 제한하여 위험을 최소화** 한다.

[세션 기반의 인증과 토큰 기반의 인증 비교]





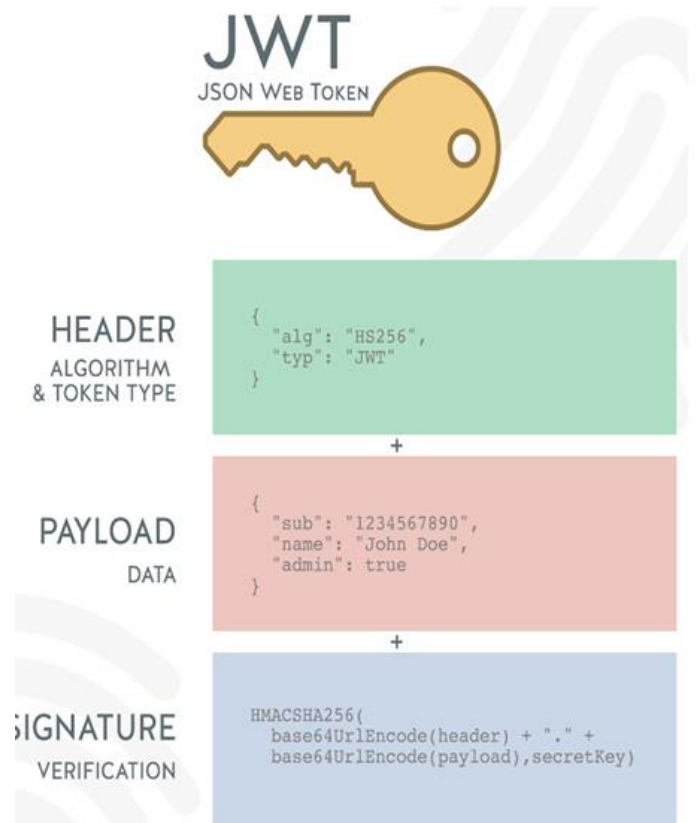
JWT (JSON Web Token) 이란

JWT(JSON Web Token)란 인증에 필요한 정보들을 암호화 시킨 JSON 토큰을 의미한다. 그리고 JWT 기반 인증은 JWT 토큰(Access Token)을 HTTP 헤더에 포함하여 전송함으로써 서버가 클라이언트를 식별하는 방식이다

JWT는 JSON 데이터를 Base64 URL-safe Encode 를 통해 인코딩하여 직렬화한 것이며, 토큰 내부에는 위변조 방지를 위해 개인키를 통한 전자서명도 들어있다. 따라서 사용자가 JWT 를 서버로 전송하면 서버는 서명을 검증하는 과정을 거치게 되며 검증이 완료되면 요청한 응답을 돌려준다.

header payload

XXXXXX.YYYYYY.Z



헤더(Header): 헤더는 토큰의 타입과 사용된 알고리즘에 대한 정보를 담고 있다. 알고리즘은 HS256이나 RS256과 같은 암호화 방식을 나타내며, 토큰 타입으로는 'JWT'가 일반적으로 사용된다.

페이로드(Payload): 페이로드는 토큰에 담길 실제 정보(클레임)를 포함한다. 여기에는 사용자의 정보, 토큰의 유효 기간, 발행자 등 다양한 클레임이 포함될 수 있다. 클레임의 종류에는 등록된 클레임, 공개 클레임, 비공개 클레임이 있으며, 각각의 목적과 사용 방법이 다르다.

서명(Signature): 서명은 헤더와 페이로드를 합친 후, 비밀키나 공개키/개인키 쌍을 사용하여 암호화하는 과정을 거친다. 이 서명을 통해 JWT의 무결성과 인증이 보장되며 JWT가 변조되지 않았음을 증명하는 용도로 사용된다. 서버는 이를 검증하여 안전한 정보 교환을 보장한다.

디코딩된 JWT는 다음과 같은 구조를 갖는다.

[Header]

```
{
  "typ": "JWT",
  "alg": "HS512"
}
```

alg : 서명 암호화 알고리즘(ex: HMAC SHA256, RSA)

typ : 토큰 유형

[Payload]

key-value 형식으로 이루어진 한 쌍의 정보를 Claim이라고 한다.

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

key-value 형식으로 이루어진 한 쌍의 정보 -

페이로드는 정해진 데이터 타입은 없지만, 대표적으로 **Registered claims**, **Public claims**, **Private claims** 이렇게 세 가지로 나뉜다.

```
{
  "iss": "growthplatform",
  "sub": "bizspring",
  "iat": 1702189795,
  "exp": 1702193395,
  "roles": "ROLE_SUPER"
}
```

Registered claims : 미리 정의된 클레임.

iss(issuer: 발행자),
exp(expiration time: 만료 시간),
sub(subject: 제목),
iat(issued At: 발행 시간),
jti(JWI ID: 토큰의 고유 식별자)

Public claims : 사용자가 정의할 수 있는 클레임 공개용 정보 전달을 위해 사용.

Private claims : 해당하는 당사자들 간에 정보를 공유하기 위해 만들어진 사용자 지정 클레임.

외부에 공개되어도 상관없지만 해당 유저를 특정할 수 있는 정보들을 담는다

[Signature]

Signature = Base64Url(Header) + . + Base64Url(PayLoad) + server's key

HS256

Header와 Payload는 단순히 인코딩된 값이기 때문에 3자가 복호화 및 조작할 수 있지만, Signature는 서버 측에서 관리하는 비밀키가 유출되지 않는 이상 복호화할 수 없다.

따라서 Signature는 토큰의 위변조 여부를 확인하는데 사용된다.

1 eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MzkwMjQsInR5cCI6IkpXVCJ9.XbPfbIHMI6arZ3Y922BhjWgQzWXcXNr0ogtVhfEd2o 2 3

1

Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2

Payload

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

3

Signature

```
HMACSHA256(
  BASE64URL(header)
  .
  BASE64URL(payload) ,
  secret)
```

토큰은 아래처럼 헤더의 Authorization 필드에 담아서 보내진다.

Authorization: <type> <credentials>

[다양한 인증 타입(알고리즘)]

토큰에는 많은 종류가 있고, 다양한 종류의 토큰을 처리하기 위해 전송받은 type에 따라 토큰을 다르게 처리한다. type에는 아래와 같은 타입들이 존재한다.

- Basic

사용자 아이디와 암호를 Base64로 인코딩한 값을 토큰으로 사용한다. (RFC 7617)

- Bearer

JWT 혹은 OAuth에 대한 토큰을 사용한다. (RFC 6750)

- Digest

서버에서 난수 데이터 문자열을 클라이언트에 보낸다. 클라이언트는 사용자 정보와 nonce를 포함하는 해시값을 사용하여 응답한다 (RFC 7616)

- HOBA

전자 서명 기반 인증 (RFC 7486)

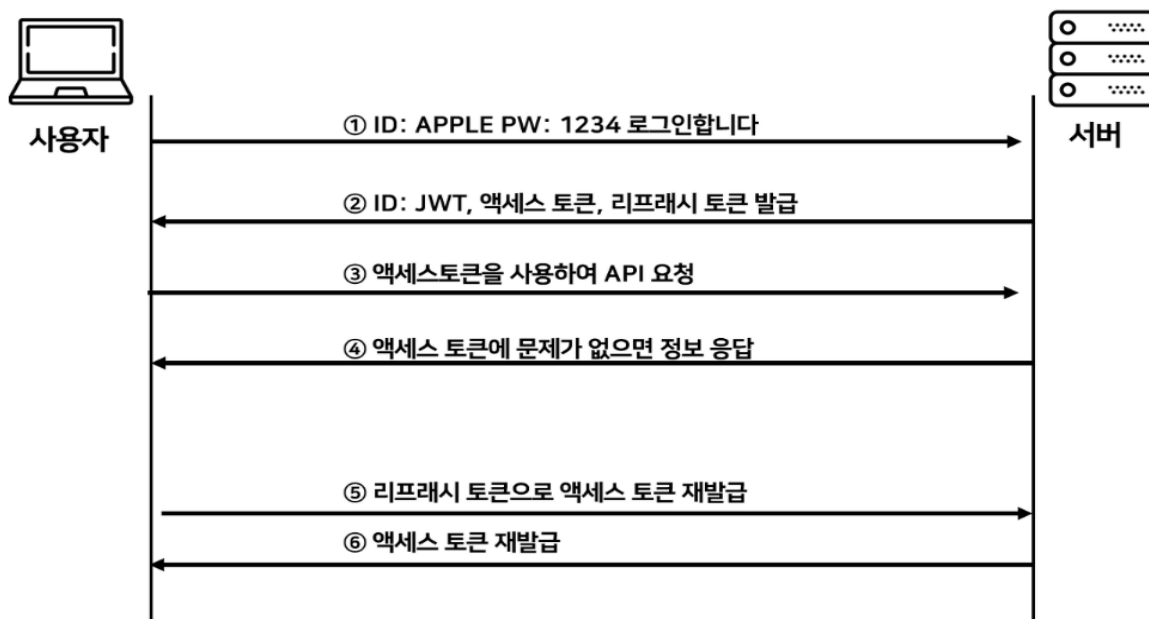
- Mutual

암호를 이용한 클라이언트-서버 상호 인증 (draft-ietf-httpauth-mutual)

- AWS4-HMAC-SHA256

AWS 전자 서명 기반 인증

JWT를 이용한 인증 과정



(1) 사용자가 ID, PW를 입력하여 서버에 로그인 인증을 요청한다.

(2) 서버에서 클라이언트로부터 인증 요청을 받으면, Header, Payload, Signature를 정의한다.

Header, Payload, Signature를 각각 Base64로 한 번 더 암호화하여 JWT를 생성하고

이를 쿠키에 담아(응답 헤더) 클라이언트에게 발급한다.

클라이언트는 서버로부터 받은 JWT를 로컬 스토리지에 저장한다.

(쿠키나 다른 곳에 저장할 수도 있음)

(3) API를 서버에 요청할 때 Authorization header에 Access Token을 담아서 보낸다.

클라이언트가 Header에 담아서 보낸 JWT가 서버에서 발행한 토큰인지 일치 여부를 확인하여

일치한다면 인증을 통과시켜주고 아니라면 통과시키지 않는 인가 과정을 수행한다.

- (4) 인증이 통과되었으므로 페이로드에 들어있는 유저의 정보들을 select해서 클라이언트가 요청한 콘텐츠를 응답한다.
- (5) 클라이언트가 서버에 요청을 했는데, 만일 액세스 토큰의 시간이 만료되면 클라이언트는 리프래시 토큰을 이용해서 서버로부터 새로운 액세스 토큰을 발급 받는다.

[토큰 인증이 신뢰성을 가지는 이유]

JWT 토큰이 A(Header) + B(Payload) + C(Signature) 일 때 (만일 임의의 유저가 B를 수정했다고 하고 B'로 표시한다.)

다른 유저가 B를 임의로 수정 -> 유저 JWT: A + B' + C

수정한 토큰을 서버에 요청을 보내면 서버는 유효성 검사 시행

유저 JWT: A + B' + C

서버에서 검증 후 생성한 JWT: A + B' + C' => (signature) 불일치

대조 결과가 일치하지 않아 유저의 정보가 임의로 조작되었음을 알 수 있다.

서버는 토큰 안에 들어있는 정보가 무엇인지 아는 게 중요한 것이 아니라 해당 토큰이 유효한 토큰인지 확인하는 것이 중요하기 때문에, 클라이언트로부터 받은 JWT의 헤더, 페이로드를 서버의 key값을 이용해 시그니처를 다시 만들고 이를 비교하며 일치했을 경우 인증을 통과시킨다.

🔗 JWT은 서명(인증)이 목적이다.

JWT는 Base64로 암호화를 하기 때문에 디버거를 사용해서 인코딩된 JWT를 1초만에 복호화할 수 있다. 복호화 하면 사용자의 데이터를 담은 Payload 부분이 그대로 노출되어 버린다. 그래서 페이로드에는 비밀번호와 같은 민감한 정보는 넣지 말아야 한다. 토큰의 진짜 목적은 정보 보호가 아닌, 위조 방지이다. 시그니처에 사용된 비밀키가 노출되지 않는 이상 데이터를 위조해도 시그니처 부분에서 바로 걸러지기 때문이다.

[JWT 장점]

- Header와 Payload를 가지고 Signature를 생성하므로 데이터 위변조를 막을 수 있다.
- 인증 정보에 대한 별도의 저장소가 필요 없다.
- JWT는 토큰에 대한 기본 정보와 전달할 정보 및 토큰이 검증됐음을 증명하는 서명 등 필요한 모든 정보를 자체적으로 가지고 있다.
- 클라이언트 인증 정보를 저장하는 세션과 다르게, 서버는 무상태(StateLess)가 되어 서버 확장성이 우수해질 수 있다.
- 토큰 기반으로 다른 로그인 시스템에 접근 및 권한 공유가 가능하다. (쿠키와 차이)
- OAuth(Open Authorization)의 경우 Facebook, Google 등 소셜 계정을 이용하여 다른 웹서비스에서도 로그인할 수 있다. 모바일 어플리케이션 환경에서도 잘 동작한다. (모바일은 세션 사용 불가능)

- 참고

서버에서 가장 피해야 할 것은 데이터베이스 조회이다.

서버 자체가 죽는 경우도 있지만, 대부분 DB가 터져서 서버도 같이 죽는 경우가 허다하기 때문이다.

이런 점에서, JWT 토큰은 DB조회를 하지 않아도 되는 장점을 가지고 있다는 점이다.

만일 payload에 유저이름과 유저등급을 함께 구성해서 보내면, 서버에서는 유저이름을 가지고 DB를 조회해서 유저 등급을 얻지 않아도 바로 원하는 정보를 취할 수 있다.

[JWT 단점]

- Self-contained : 토큰 자체에 정보를 담고 있으므로 양날의 검이 될 수 있다.
- 토큰 길이 : 토큰의 Payload에 3종류의 클레임을 저장하기 때문에, 정보가 많아질수록 토큰의 길이가 늘어나 네트워크에 부하를 줄 수 있다.
- Payload 인코딩 : payload 자체는 암호화 된 것이 아니라 BASE64로 인코딩 된 것이기 때문에, 중간에 Payload를 탈취하여 디코딩하면 데이터를 볼 수 있으므로, payload에 중요 데이터를 넣지 않아야 한다.
- Store Token : stateless 특징을 가지기 때문에, 토큰은 클라이언트 측에서 관리하고 저장한다. 때문에 토큰 자체를 탈취당하면 대처하기가 어렵게 된다.

[JWT를 사용한 이유]

모바일 앱 JWT가 사용된 주 이유는 결국 모바일 앱의 등장이다. 모바일 앱의 특성상 주로 JWT 방식으로 인증/인가를 진행한다. 모바일 앱에서의 로그아웃 모바일 앱에서는 JWT 탈취 우려가 거의 없기 때문에 앱단에서 로그아웃을 진행하여 JWT 자체를 제거해버리면 서버측에선 추가 조치도 필요가 없다. 장기간 동안 로그인 상태를 유지하려고 세션 설정을 하면 서버 측 부하가 많이 가기 때문에 JWT 방식을 이용하는 것도 한 방법이다.

[JWT의 Access Token / Refresh Token]

JWT도 제 3자에게 토큰 탈취의 위험성이 있기 때문에, 그대로 사용하는 것이 아닌 Access Token, Refresh Token으로 이중으로 나누어 인증을 하는 방식을 현업에선 취한다.

Access Token 과 Refresh Token은 모두 똑같은 JWT이다. 다만 토큰이 어디에 저장되고 관리 되느냐에 따른 사용 차이일 뿐이다.

Access Token : 클라이언트가 가지고 있는 사용자의 정보가 담긴 토큰으로, 클라이언트에서 요청이 오면 서버에서는 해당 토큰에 있는 정보를 활용하여 사용자 정보에 맞게 응답 진행

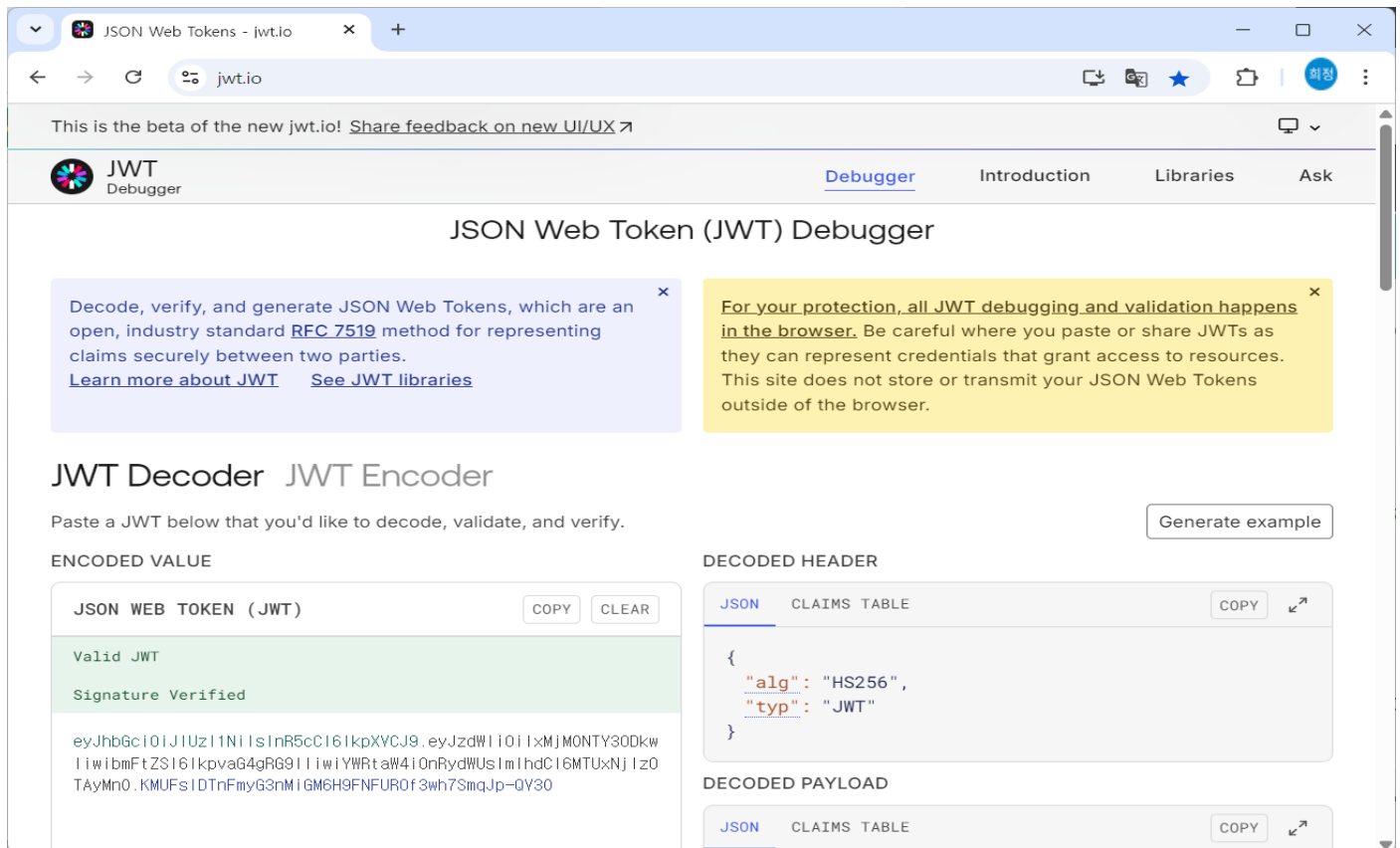
Refresh Token: 새로운 Access Token을 발급해주기 위해 사용하는 토큰으로 짧은 수명을 가지는 Access Token에 게 새로운 토큰을 발급해주기 위해 사용. 해당 토큰은 보통 데이터베이스에 유저 정보와 같이 기록.

Access Token은 접근에 관여하는 토큰, Refresh Token은 재발급에 관여하는 토큰의 역할로 사용되는 JWT 라고 할 수 있다.

[JWT 인코딩 / 디코딩 해보기]

직접 JWT 토큰을 생성해 연습해보고 싶다면, 아래 공식사이트에서 쉽게 JWT 토큰을 인코딩(생성) 하거나 디코딩 할 수 있다.

<https://jwt.io/>



[JWT를 통한 인증절차]

- 사용자가 로그인을 한다.
- 서버에서는 계정 정보를 읽어 사용자를 확인한 후, 사용자의 고유 ID 값을 부여하고 기타 정보와 함께 Payload 에 집어넣는다.
- JWT 토큰의 유효기간을 설정한다.

암호화할 **Secret key** 를 이용해 Access Token 을 발급한다. 클라이언트는 Access Token 을 받아 저장 후, 인증이 필요한 요청마다 토큰을 헤더에 실어 보낸다.

서버에서는 해당 토큰의 **Verify Signature** 를 **Secret key** 로 복호화한 후, 조작 여부, 유효기간을 **확인**한다. 검증이 완료되었을 경우, Payload 를 디코딩 하여 사용자의 ID 에 맞는 데이터를 가져온다. 보통 Access Token의 유효기간은 매우 짧게 설정한다. Refresh Token을 따로 발급해서 Access Token 이 만료되면 새로운 JWT를 발급할 수 있게 한다.

[Refresh Token]

Refresh Token이란 Access Token을 재발급 받기위한 Token이다.

1. 클라이언트에서 로그인한다.
2. 서버는 클라이언트에게 Access Token과 Refresh Token을 발급한다. 동시에 Refresh Token은 서버에도 저장된다. 클라이언트는 local 저장소에 두 Token을 저장한다.
3. 매 요청마다 Access Token을 헤더에 담아서 전달한다.
4. 이 때, Access Token이 만료가 되면 서버는 만료되었다는 Response를 하게 된다.
5. 클라이언트는 해당 Response를 받으면 Refresh Token을 보낸다.
6. 서버는 Refresh Token 유효성 체크를 하게 되고, 새로운 Access Token을 발급한다.
7. 클라이언트는 새롭게 받은 Access Token을 기존의 Access Token에 덮어쓰게 된다.

예) Access Token 만료기간 : 30분 ~ 1시간, Refresh Token 만료기간 : 3일 ~ 1달

